

Sistemas embebidos

Instrucciones de aritmético lógicas

μ C ATmega328P

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Instrucciones Aritmético-Lógicas

Las instrucciones aritmético-lógicas realizan **operaciones matemáticas y lógicas** sobre los datos **almacenados en registros**.

Categorías principales:

- **Operaciones Aritméticas:** Suma, resta, multiplicación
- **Operaciones Lógicas:** AND, OR, XOR, NOT
- **Operaciones de Desplazamiento:** Rotación y shifts
- **Operaciones de Comparación:** Comparar valores
- **Operaciones de Bit:** Manipulación individual de bits

Importante

Todas estas operaciones afectan el **registro de estado (SREG)**

Registro de Estado (SREG)

El **SREG** contiene **banderas** que reflejan el **resultado** de las **operaciones**

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
I	T	H	S	V	N	Z	C

Significado de las banderas:

- **I (Interrupt):** Habilitación global de **interrupciones**
- **T (Transfer):** Bit de transferencia
- **H (Half Carry):** Acarreo del **bit 3**
- **S (Sign):** Signo ($N \oplus V$)
- **V (Overflow):** Desbordamiento en **complemento a 2**
- **N (Negative):** Resultado **negativo**
- **Z (Zero):** Resultado **cero**
- **C (Carry):** Acarreo

Instrucciones de Suma

ADD - Add without Carry

ADD Rd, Rr - Suma dos registros **sin acarreo**

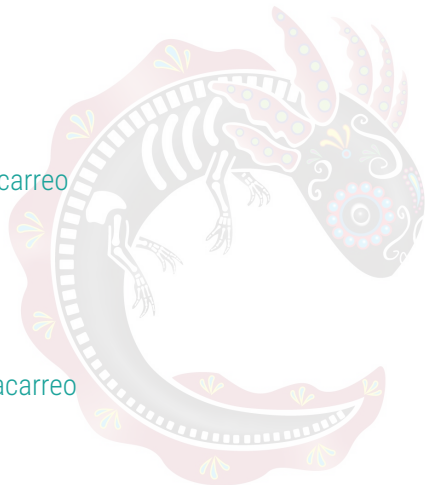
Operación: $Rd \leftarrow Rd + Rr$

Banderas afectadas: **H**, **S**, **V**, **N**, **Z**, **C**

ADC - Add with Carry

ADC Rd, Rr - Suma dos registros **con acarreo**

Operación: $Rd \leftarrow Rd + Rr + C$



Instrucciones de Suma

Ejemplos:

```
1 LDI r16, 0x15          ; Cargar 21
2 LDI r17, 0x28          ; Cargar 40
3 ADD r16, r17            ; r16 = 21 + 40 = 61 (0x3D)
4
5 ; Suma de 16 bits (R17:R16 + R19:R18)
6 ADD r16, r18            ; Parte baja
7 ADC r17, r19            ; Parte alta + carry
```

Instrucciones de Resta

SUB - Subtract without Carry

SUB R_d , R_r - Resta dos registros **sin préstamo**

Operación: $R_d \leftarrow R_d - R_r$

SBC - Subtract with Carry

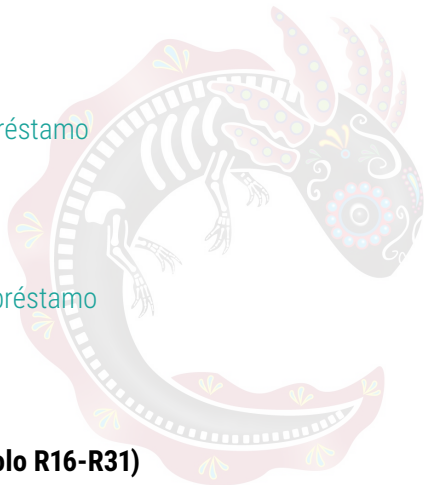
SBC R_d , R_r - Resta dos registros **con préstamo**

Operación: $R_d \leftarrow R_d - R_r - C$

SUBI - Subtract Immediate

SUBI R_d , k - Resta **valor inmediato (solo R16-R31)**

Operación: $R_d \leftarrow R_d - k$



Instrucciones de Resta

Ejemplos:

```
1 LDI r16, 0x50          ; Cargar 80
2 LDI r17, 0x25          ; Cargar 37
3 SUB r16, r17           ; r16 = 80 - 37 = 43 (0x2B)
4
5 SUBI r16, 0x10         ; r16 = r16 - 16
```

Incremento y Decremento

INC - Increment

INC Rd - **Incrementa** registro en 1

Operación: $Rd \leftarrow Rd + 1$

Banderas afectadas: **S**, **V**, **N**, **Z** (NO afecta **C**)

DEC - Decrement

DEC Rd - **Decrementa** registro en 1

Operación: $Rd \leftarrow Rd - 1$

Banderas afectadas: **S**, **V**, **N**, **Z** (NO afecta **C**)



Incremento y Decremento

Ejemplos:

```
1 LDI r16, 0x00          ; Cargar 0
2 INC r16                 ; r16 = 1
3 INC r16                 ; r16 = 2
4
5 LDI r17, 0x05           ; Cargar 5
6 loop:
7 DEC r17                 ; Decrementar
8 BRNE loop              ; Repetir si no es cero
```

Operaciones de Multiplicación

MUL - Multiply Unsigned

MUL Rd, Rr - Multiplicación **sin signo**

Operación: $R1:R0 \leftarrow Rd \times Rr$ (**resultado de 16 bits**)

MULS - Multiply Signed

MULS Rd, Rr - Multiplicación **con signo** (**solo R16-R31**)

Operación: $R1:R0 \leftarrow Rd \times Rr$ (con signo)

MULSU - Multiply Signed with Unsigned

MULSU Rd, Rr - Rd **con signo**, Rr **sin signo** (**solo R16-R23**)



Operaciones de Multiplicación

Ejemplo:

```
1 LDI r16, 15           ; Multiplicando
2 LDI r17, 12           ; Multiplicador
3 MUL r16, r17          ; 15 * 12 = 180
4 ; Resultado: R1 = 0x00, R0 = 0xB4 (180 = 0x00B4)
5
6 MOV r18, r0           ; Copiar parte baja
7 MOV r19, r1           ; Copiar parte alta
```

Ejemplo: Suma de 32 bits

```

1 ; Numero A en R19:R18:R17:R16 (MSB:LSB)
2 ; Numero B en R23:R22:R21:R20 (MSB:LSB)
3 ; Resultado en R19:R18:R17:R16
4
5 suma_32bits:
6     CLC                ; Limpiar carry
7 ; Sumar byte menos significativo
8     ADD r16, r20        ; LSB
9 ; Sumar segundo byte con carry
10    ADC r17, r21
11 ; Sumar tercer byte con carry
12    ADC r18, r22
13 ; Sumar byte mas significativo con carry
14    ADC r19, r23        ; MSB
15 ; Verificar si hubo overflow
16    BRCS overflow_error ; Saltar si C=1
17    RET
18
19 ; Manejar error de desbordamiento
20 overflow_error:
21    LDI r16, 0xFF        ; Codigo de error
22    RET

```

Operaciones AND, OR, XOR

AND - Logical AND

AND Rd, Rr - AND lógico entre dos registros

Operación: $Rd \leftarrow Rd \wedge Rr$

OR - Logical OR

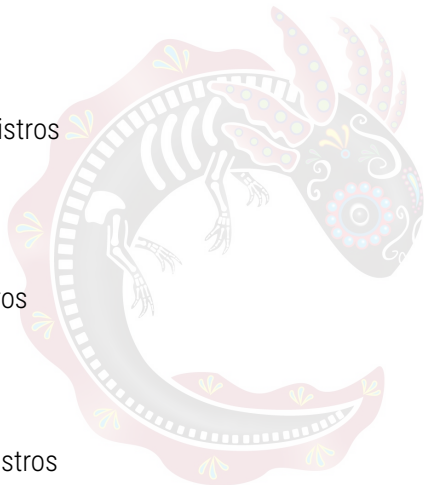
OR Rd, Rr - OR lógico entre dos registros

Operación: $Rd \leftarrow Rd \vee Rr$

EOR - Exclusive OR

EOR Rd, Rr - XOR lógico entre dos registros

Operación: $Rd \leftarrow Rd \oplus Rr$



Operaciones AND, OR, XOR

Ejemplos:

```
1 LDI r16, 0b11110000 ; Cargar patron
2 LDI r17, 0b10101010 ; Cargar mascara
3
4 AND r16, r17          ; r16 = 0b10100000 (mascarar bits)
5 OR  r16, r17          ; r16 = r16 OR r17
6 EOR r16, r17          ; r16 = r16 XOR r17 (toggle bits)
```

Operaciones AND, OR con Inmediato

ANDI - Logical AND with Immediate

ANDI Rd, k - AND lógico con valor inmediato (solo R16-R31)

Operación: $Rd \leftarrow Rd \wedge k$

ORI - Logical OR with Immediate

ORI Rd, k - OR lógico con valor inmediato (solo R16-R31)

Operación: $Rd \leftarrow Rd \vee k$



Operaciones AND, OR con Inmediato

Ejemplos comunes:

```
1 ; Limpiar bits especificos (mascara AND)
2 LDI r16, 0b11111111
3 ANDI r16, 0b11110000 ; Limpiar bits 0-3
4
5 ; Establecer bits especificos (mascara OR)
6 LDI r17, 0b00000000
7 ORI r17, 0b00001111 ; Establecer bits 0-3
8
9 ; Alternar bits especificos (mascara XOR)
10 LDI r18, 0b10101010
11 LDI r19, 0b11111000
12 EOR r18, r19 ; Alternar bits segun mascara
```


Complemento y Negación

COM - One's Complement

COM Rd - Complemento a uno (invertir todos los bits)

Operación: $Rd \leftarrow 0xFF - Rd$

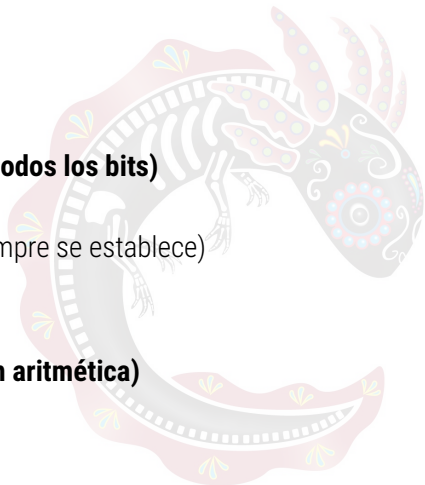
Banderas afectadas: S, V, N, Z, C (C siempre se establece)

NEG - Two's Complement

NEG Rd - Complemento a dos (negación aritmética)

Operación: $Rd \leftarrow 0x00 - Rd$

Banderas afectadas: H, S, V, N, Z, C



Complemento y Negación

Ejemplos:

```
1 LDI r16, 0b10101010 ; Cargar patron
2 COM r16              ; r16 = 0b01010101 (invertir)
3
4 LDI r17, 5           ; Cargar 5
5 NEG r17              ; r17 = -5 (0xFB en complemento a 2)
6
7 LDI r18, 0x80        ; -128 en complemento a 2
8 NEG r18              ; r18 = 0x80 (overflow!)
```

Manipulación de Bits Individuales

SBR - Set Bits in Register

SBR Rd, k - **Establecer** bits (equivalente a ORI)

CBR - Clear Bits in Register

CBR Rd, k - **Limpiar** bits (equivalente a ANDI Rd, (0xFF-k))

BST - Bit Store from Register to T

BST Rr, b - **Copiar bit** del registro **a la bandera T**

BLD - Bit Load from T to Register

BLD Rd, b - **Copiar bandera T** a un bit del **registro**



Ejemplos:

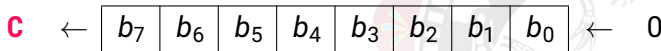
```
1 LDI r16, 0x00
2 SBR r16, 0b00001111 ; Establecer bits 0-3
3 CBR r16, 0b11110000 ; Limpiar bits 4-7
4
5 BST r16, 3 ; Copiar bit 3 de r16 a T
6 BLD r17, 7 ; Copiar T a bit 7 de r17
```

Desplazamientos Lógicos

LSL - Logical Shift Left

LSL Rd - Desplazamiento lógico a la izquierda

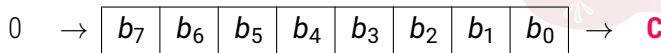
Operación:



LSR - Logical Shift Right

LSR Rd - Desplazamiento lógico a la derecha

Operación:



Desplazamientos Lógicos

Ejemplos:

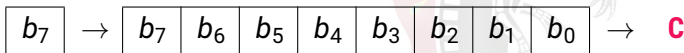
```
1 LDI r16, 0b10110001 ; Cargar valor
2 LSL r16              ; r16 = 0b01100010, C = 1
3 LSL r16              ; r16 = 0b11000100, C = 0
4
5 LDI r17, 0b10110001 ; Cargar valor
6 LSR r17              ; r17 = 0b01011000, C = 1
7 LSR r17              ; r17 = 0b00101100, C = 0
```

Desplazamiento Aritmético

ASR - Arithmetic Shift Right

ASR Rd - Desplazamiento aritmético a la derecha

Operación:

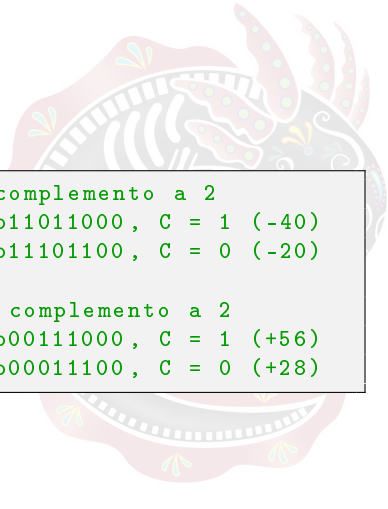


Características:

- Preserva el **bit de signo (MSB)**
- Útil para **división por 2** con signo
- El **LSB** se mueve al **carry**

Desplazamiento Aritmético

Ejemplos:



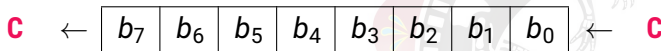
```
1 LDI r16, 0b10110001 ; -79 en complemento a 2
2 ASR r16 ; r16 = 0b11011000, C = 1 (-40)
3 ASR r16 ; r16 = 0b11101100, C = 0 (-20)
4
5 LDI r17, 0b01110001 ; +113 en complemento a 2
6 ASR r17 ; r17 = 0b00111000, C = 1 (+56)
7 ASR r17 ; r17 = 0b00011100, C = 0 (+28)
```


Rotaciones

ROL - Rotate Left through Carry

ROL Rd - Rotación a la izquierda a través del carry

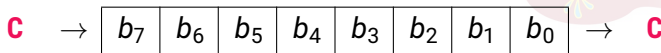
Operación:



ROR - Rotate Right through Carry

ROR Rd - Rotación a la derecha a través del carry

Operación:



Rotaciones

Ejemplos:

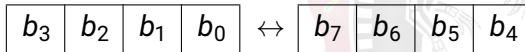
```
1 LDI r16, 0b10110001 ; Cargar valor
2 CLC                  ; Limpiar carry (C = 0)
3 ROL r16              ; r16 = 0b01100010, C = 1
4 ROL r16              ; r16 = 0b11000101, C = 0
5
6 LDI r17, 0b10110001 ; Cargar valor
7 SEC                  ; Establecer carry (C = 1)
8 ROR r17              ; r17 = 0b11011000, C = 1
9 ROR r17              ; r17 = 0b11101100, C = 0
```

SWAP - Intercambiar Nibbles

SWAP - Swap Nibbles

SWAP Rd - Intercambia los nibbles alto y bajo

Operación:



Características:

- No afecta **ninguna bandera** del **SREG**
- Útil para **manipulación** de **datos BCD**
- Operación **reversible** (SWAP dos veces restaura el valor original)

SWAP - Intercambiar Nibbles

Ejemplos:

```
1 LDI r16, 0xAB          ; Cargar 0xAB (10101011)
2 SWAP r16               ; r16 = 0xBA (10111010)
3 SWAP r16               ; r16 = 0xAB (valor original)
4
5 ; Util para conversiones BCD
6 LDI r17, 0x47          ; BCD: 47
7 SWAP r17               ; r17 = 0x74
8 ANDI r17, 0x0F         ; r17 = 0x04 (digito alto extraido)
```

Instrucciones de Comparación

CP - Compare

CP Rd, Rr - Compara dos registros (Rd - Rr)

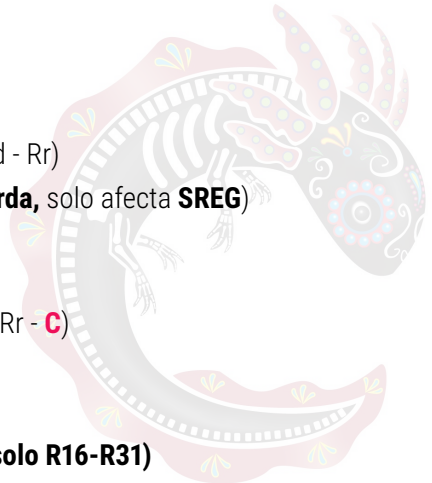
Operación: Rd - Rr (resultado **no se guarda**, solo afecta **SREG**)

CPC - Compare with Carry

CPC Rd, Rr - Compara con carry (Rd - Rr - C)

CPI - Compare with Immediate

CPI Rd, k - Compara con inmediato (**solo R16-R31**)



Instrucciones de Comparación

Ejemplos:

```
1 LDI r16, 25           ; Cargar 25
2 LDI r17, 30           ; Cargar 30
3 CP r16, r17           ; Comparar: 25 - 30
4 BRLT menor           ; Saltar si r16 < r17 (Z=0, S=1)
5 BREQ igual           ; Saltar si r16 = r17 (Z=1)
6 BRGE mayor_igual     ; Saltar si r16 >= r17 (S=0)
7
8 CPI r16, 20           ; Comparar r16 con 20
9 BRSH mayor_igual_20  ; Saltar si r16 >= 20 (sin signo)
```

Comparación y Salto Condicional

CPSE - Compare, Skip if Equal

CPSE Rd, Rr - Compara y salta si son iguales

Operación: Si $Rd = Rr$, **salta** la siguiente instrucción

TST - Test for Zero or Minus

TST Rd - Prueba si es **cero** o **negativo** (equivale a AND Rd, Rd)

Ejemplos:

```

1 LDI r16, 10
2 LDI r17, 10
3 CPSE r16, r17           ; r16 = r17?
4 RJMP no_iguales        ; Se salta si son iguales
5 ; Aquí se ejecuta si son iguales
6
7 TST r16                 ; Probar r16
8 BRMI negativo          ; Saltar si N=1 (negativo)
9 BREQ cero              ; Saltar si Z=1 (cero)

```

Ejemplo 1: Suma BCD

```

1 ; Sumar dos numeros BCD de 2 digitos
2 ; Numero A en r16 (ej: 0x47 = 47 decimal)
3 ; Numero B en r17 (ej: 0x38 = 38 decimal)
4 ; Resultado en r18
5
6 suma_bcd:
7 ; Extraer digitos bajos
8     MOV r19, r16
9     ANDI r19, 0x0F      ; Digito bajo de A
10    MOV r20, r17
11    ANDI r20, 0x0F      ; Digito bajo de B
12
13 ; Sumar digitos bajos
14    ADD r19, r20
15    CPI r19, 10          ; Resultado >= 10?
16    BRLO sin_ajuste_bajo
17    SUBI r19, 10          ; Ajustar si >= 10
18    SEC                  ; Establecer carry
19    RJMP siguiente
20
21 sin_ajuste_bajo:
22    CLC                  ; Limpiar carry

```


Ejemplo 1: Suma BCD

```

1 siguiente:
2 ; Extraer digitos altos
3     MOV r21, r16
4     SWAP r21
5     ANDI r21, 0x0F      ; Dígito alto de A
6     MOV r22, r17
7     SWAP r22
8     ANDI r22, 0x0F      ; Dígito alto de B
9
10 ; Sumar digitos altos + carry
11     ADC r21, r22
12     CPI r21, 10        ; Resultado >= 10?
13     BRLO sin_ajuste_alto
14     SUBI r21, 10        ; Ajustar si >= 10
15 ; Aquí habria overflow en BCD de 2 digitos
16
17 sin_ajuste_alto:
18 ; Combinar resultado
19     SWAP r21            ; Mover dígito alto a posición
20     OR r21, r19         ; Combinar con dígito bajo
21     MOV r18, r21        ; Guardar resultado
22     RET

```

Ejemplo 2: Multiplicación por Potencias de 2

```
1 ; Multiplicar por potencias de 2 usando LSL
2 ; r16 contiene el valor a multiplicar
3
4 multiplicar_x2:
5     LSL r16                ; Multiplicar por 2
6     RET
7
8 multiplicar_x4:
9     LSL r16                ; x2
10    LSL r16                ; x4
11    RET
12
13 multiplicar_x8:
14    LSL r16                ; x2
15    LSL r16                ; x4
16    LSL r16                ; x8
17    RET
18
19 ; Multiplicar por 10 ( $2*5 = 2*(4+1) = 8+2$ )
20 multiplicar_x10:
21    MOV r17, r16           ; Copiar valor original
22    LSL r16                ; x2
```

Ejemplo 2: Multiplicación por Potencias de 2

```
1      LSL r16                ; x4
2      LSL r16                ; x8
3      ADD r16, r17           ; 8*n + n = 9*n
4      ADD r16, r17           ; 9*n + n = 10*n
5      RET
6
7      ; Division por potencias de 2 usando LSR
8      dividir_x2:
9          LSR r16             ; Dividir por 2 (sin signo)
10         RET
11
12     dividir_x2_con_signo:
13         ASR r16             ; Dividir por 2 (con signo)
14         RET
```

Ejemplo 3: Conversión ASCII a Binario

```

1  ; Convertir digito ASCII (r16) a valor binario (r17)
2  ; Entrada: r16 = '0' a '9' (0x30 a 0x39)
3  ; Salida: r17 = 0 a 9, r18 = codigo de error
4
5  ascii_a_binario:
6  ; Verificar rango valido
7      CPI r16, '0'          ; Menor que '0'?
8      BRLO error_rango
9      CPI r16, '9'+1        ; Mayor que '9'?
10     BRSH error_rango
11
12 ; Conversion valida
13     MOV r17, r16
14     SUBI r17, '0'          ; Restar valor ASCII de '0'
15     CLR r18                ; Sin error
16     RET
17
18 error_rango:
19     LDI r17, 0xFF          ; Valor invalido
20     LDI r18, 0x01          ;Codigo de error
21     RET

```

Ejemplo 3: Conversión ASCII a Binario

```

1 ; Convertir numero BCD a ASCII
2 bcd_a_ascii:
3 ; Convertir nibble bajo
4     MOV r17, r16
5     ANDI r17, 0x0F      ; Extraer nibble bajo
6     SUBI r17, -'0'      ; Sumar ASCII '0' (equivale a ADDI)
7
8 ; Convertir nibble alto
9     MOV r18, r16
10    SWAP r18             ; Mover nibble alto a bajo
11    ANDI r18, 0x0F      ; Extraer nibble
12    SUBI r18, -'0'      ; Sumar ASCII '0'
13
14 ; r18 = digito alto ASCII, r17 = digito bajo ASCII
15     RET

```

Comparación de Rendimiento

Operación	Ciclos	Palabras	Alternativa
ADD/SUB	1	1	-
MUL	2	1	Desplazamientos para potencias de 2
LSL/LSR	1	1	Multiplicar/dividir por 2
AND/OR/EOR	1	1	-
COM/NEG	1	1	-
CP/CPI	1	1	SUB (si necesitas el resultado)
SWAP	1	1	4 rotaciones
INC/DEC	1	1	ADD/SUB con inmediato

Tips de Optimización

- Usar **desplazamientos** para multiplicar/dividir por potencias de 2
- **INC/DEC** no afectan carry (útil en algunas operaciones)
- **SWAP** es más rápido que 4 rotaciones
- **TST** es equivalente a CP Rd, 0 pero más claro

Técnicas de Optimización

1. Multiplicación por constantes pequeñas:

```
1 ; Multiplicar por 3: r16 = r16 * 3
2 MOV r17, r16           ; Copia
3 LSL r16                 ; r16 * 2
4 ADD r16, r17           ; r16 * 2 + r16 = r16 * 3
5
6 ; Multiplicar por 5: r16 = r16 * 5
7 MOV r17, r16           ; Copia
8 LSL r16                 ; r16 * 2
9 LSL r16                 ; r16 * 4
10 ADD r16, r17           ; r16 * 4 + r16 = r16 * 5
```

Técnicas de Optimización

2. Limpiar registro eficientemente:

```
1 ; Lento
2 LDI r16, 0x00
3
4 ; Rapido (si r16 ya tiene algun valor)
5 EOR r16, r16 ; r16 XOR r16 = 0
6 ; o
7 SUB r16, r16 ; r16 - r16 = 0
```


Resumen - Operaciones Aritméticas

Instrucción	Operación	Banderas
ADD Rd, Rr	$Rd \leftarrow Rd + Rr$	H, S, V, N, Z, C
ADC Rd, Rr	$Rd \leftarrow Rd + Rr + C$	H, S, V, N, Z, C
SUB Rd, Rr	$Rd \leftarrow Rd - Rr$	H, S, V, N, Z, C
SBC Rd, Rr	$Rd \leftarrow Rd - Rr - C$	H, S, V, N, Z, C
SUBI Rd, k	$Rd \leftarrow Rd - k$	H, S, V, N, Z, C
INC Rd	$Rd \leftarrow Rd + 1$	S, V, N, Z
DEC Rd	$Rd \leftarrow Rd - 1$	S, V, N, Z
MUL Rd, Rr	$R1:R0 \leftarrow Rd \times Rr$	Z, C
MULS Rd, Rr	$R1:R0 \leftarrow Rd \times Rr$ (signed)	Z, C
NEG Rd	$Rd \leftarrow 0 - Rd$	H, S, V, N, Z, C

Resumen - Operaciones Lógicas

Instrucción	Operación	Banderas
AND Rd, Rr	$Rd \leftarrow Rd \wedge Rr$	S,V,N,Z
ANDI Rd, k	$Rd \leftarrow Rd \wedge k$	S,V,N,Z
OR Rd, Rr	$Rd \leftarrow Rd \vee Rr$	S,V,N,Z
ORI Rd, k	$Rd \leftarrow Rd \vee k$	S,V,N,Z
EOR Rd, Rr	$Rd \leftarrow Rd \oplus Rr$	S,V,N,Z
COM Rd	$Rd \leftarrow 0xFF - Rd$	S,V,N,Z,C
SBR Rd, k	$Rd \leftarrow Rd \vee k$	S,V,N,Z
CBR Rd, k	$Rd \leftarrow Rd \wedge (0xFF - k)$	S,V,N,Z
BST Rr, b	$T \leftarrow Rr[b]$	T
BLD Rd, b	$Rd[b] \leftarrow T$	-

Resumen - Desplazamientos y Comparaciones

Desplazamientos:

Instr.	Operación
LSL Rd	C $\leftarrow$ Rd $\leftarrow$ 0
LSR Rd	0 $\rightarrow$ Rd $\rightarrow$ C
ASR Rd	MSB $\rightarrow$ Rd $\rightarrow$ C
ROL Rd	C $\leftrightarrow$ Rd
ROR Rd	C $\leftrightarrow$ Rd
SWAP Rd	Nibbles $\leftrightarrow$

Comparaciones:

Instr.	Operación
CP Rd, Rr	Rd - Rr
CPC Rd, Rr	Rd - Rr - C
CPI Rd, k	Rd - k
CPSE Rd, Rr	Skip if Rd=Rr
TST Rd	Rd $\wedge$ Rd

Puntos clave para recordar:

- Las **operaciones aritméticas** **afectan más banderas** que las lógicas
- INC/DEC** **no afectan** el carry (útil en contadores)
- Los desplazamientos son **equivalentes** a **multiplicar/dividir por 2**
- SWAP** **no afecta** ninguna bandera del **SREG**